

Computer Architecture Study

Master:

Dr. Safari

Project:

Pipelined RISC-V Implementation

Contributors:

Anita Zekavat & Amir Hosseini

1. Datapath Design and Incremental Development

In the initial phase of the project, we focused on designing the datapath based on the specified requirements. An incremental design methodology was employed, where each instruction was integrated and verified individually to ensure functional correctness at every stage. This systematic approach allowed for precise debugging of data flow and control signals. The final architecture of the datapath, optimized for multi-cycle pipeline stages, is illustrated below:

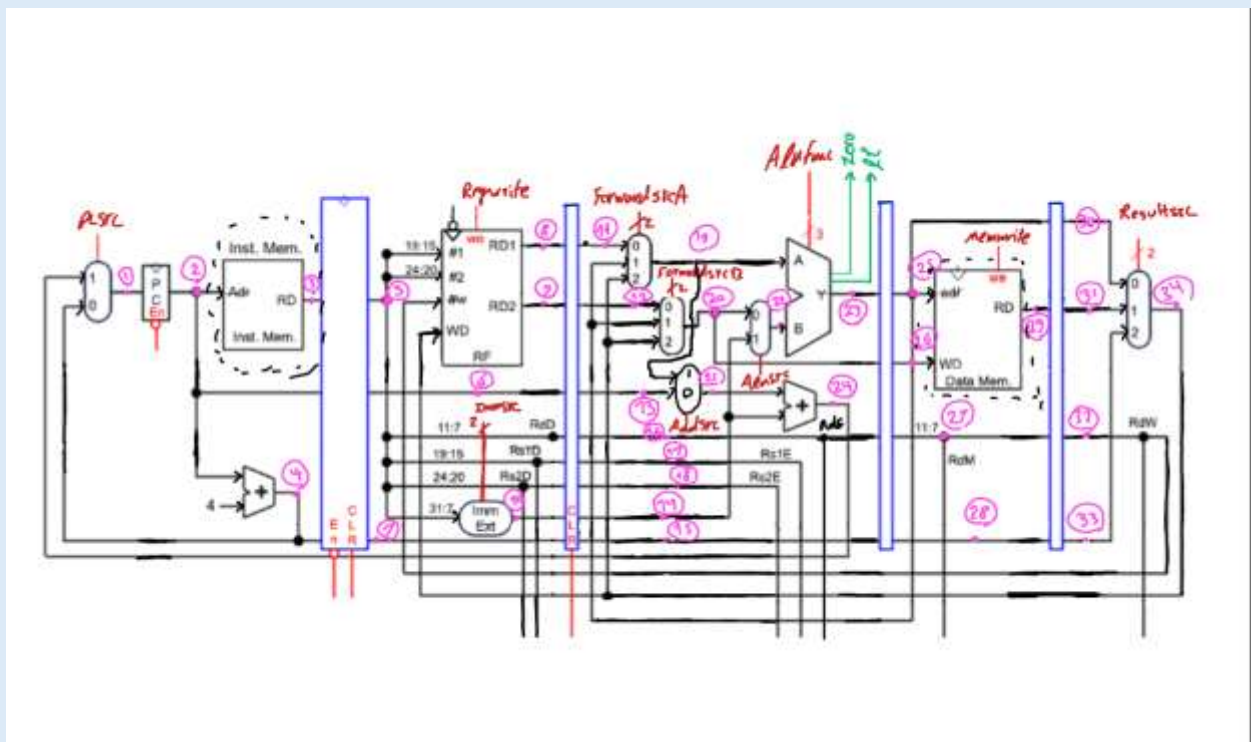


Figure 1: "\$PATH\TO\PROJECT\design\papers\Data_path_paper

The names of wires within the datapath are detailed in the accompanying table:

$IF \in [1, 4]$	$ID \in [5, 10]$	$EX \in [11, 24]$	$MEM \in [25, 29]$	$WB \in [30, 31]$
1 $\rightarrow PC_{stoutf}$	5 $\rightarrow InstRDID$	11 $\rightarrow RD1outE$	25 $\rightarrow ALUoutM$	30 $\rightarrow ALUoutW$
2 $\rightarrow PCoutf$	6 $\rightarrow PCoutD$	12 $\rightarrow RD2outE$	26 $\rightarrow ForwardoutM$	31 $\rightarrow MemRDW$
3 $\rightarrow InstRDF$	7 $\rightarrow IncoutD$	13 $\rightarrow PCoutE$	27 $\rightarrow RDM$	32 $\rightarrow RDW$
4 $\rightarrow Incoutf$	8 $\rightarrow RD1outD$	14 $\rightarrow ImmExtoutE$	28 $\rightarrow IncoutM$	33 $\rightarrow IncoutW$
	9 $\rightarrow RD2outD$	15 $\rightarrow EncoutE$	29 $\rightarrow MemRDW$	34 $\rightarrow ResultoutW$
	10 $\rightarrow ImmExtoutD$	16 $\rightarrow RDE$		
		17 $\rightarrow RS1E$		
		18 $\rightarrow RS2E$		
		19 $\rightarrow Forward/SRCoutE$		
		20 $\rightarrow Forward/SRCoutE$		
		21 $\rightarrow addSoutE$		
		22 $\rightarrow ALUSoutE$		
		23 $\rightarrow ALUoutE$		
		24 $\rightarrow AddoutE$		

Figure 2: "\$PATH\TO\PROJECT\design\papers\Data_path_paper

Additionally, the Arithmetic Logic Unit (ALU) was designed to handle the required arithmetic and logical operations. The truth table representing the ALU's operational logic and control signals is provided as follows:

A	B	
ALU func	Operation	
0	A + B	
1	A - B	
10	A & B	
11	A B	
100	A ^ B	
101	A < B	

Figure 3: "\$PATH\TO\PROJECT\design\tables\ALU_function_table

2. Control Unit and Pipeline Integration

The controller architecture was derived from a single-cycle design and subsequently adapted for the pipelined environment. To ensure synchronization with the datapath, several pipeline registers were strategically integrated, and minor modifications were made to the control logic to handle stage-specific signal propagation. The control unit is modularized into three primary components: the **Main Controller**, the **Func Controller**, and the **PC Controller**. The final schematic and the corresponding truth tables for these units are as follows:

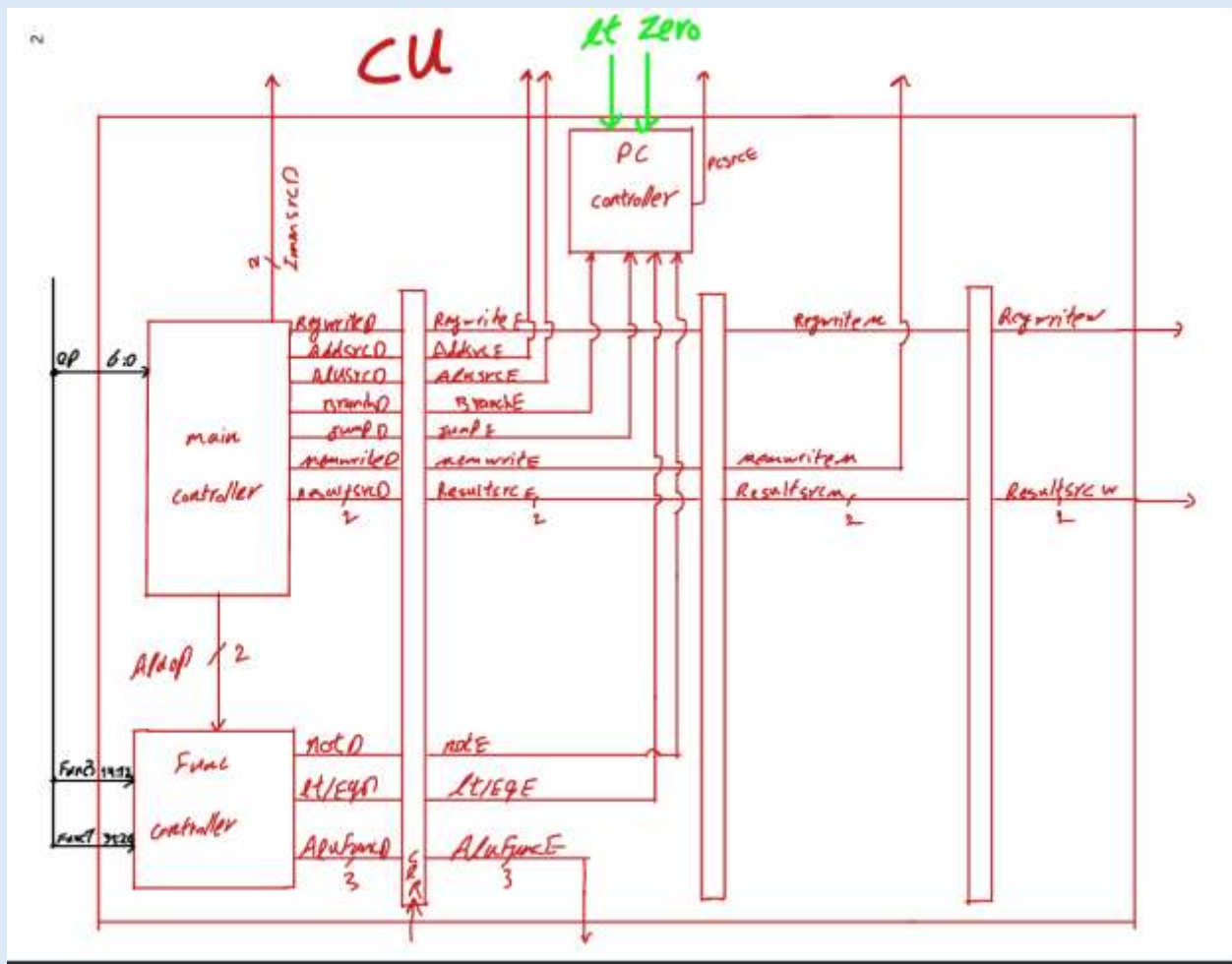


Figure 4: “\$PATH\TO\PROJECT\”design\papers\CU paper

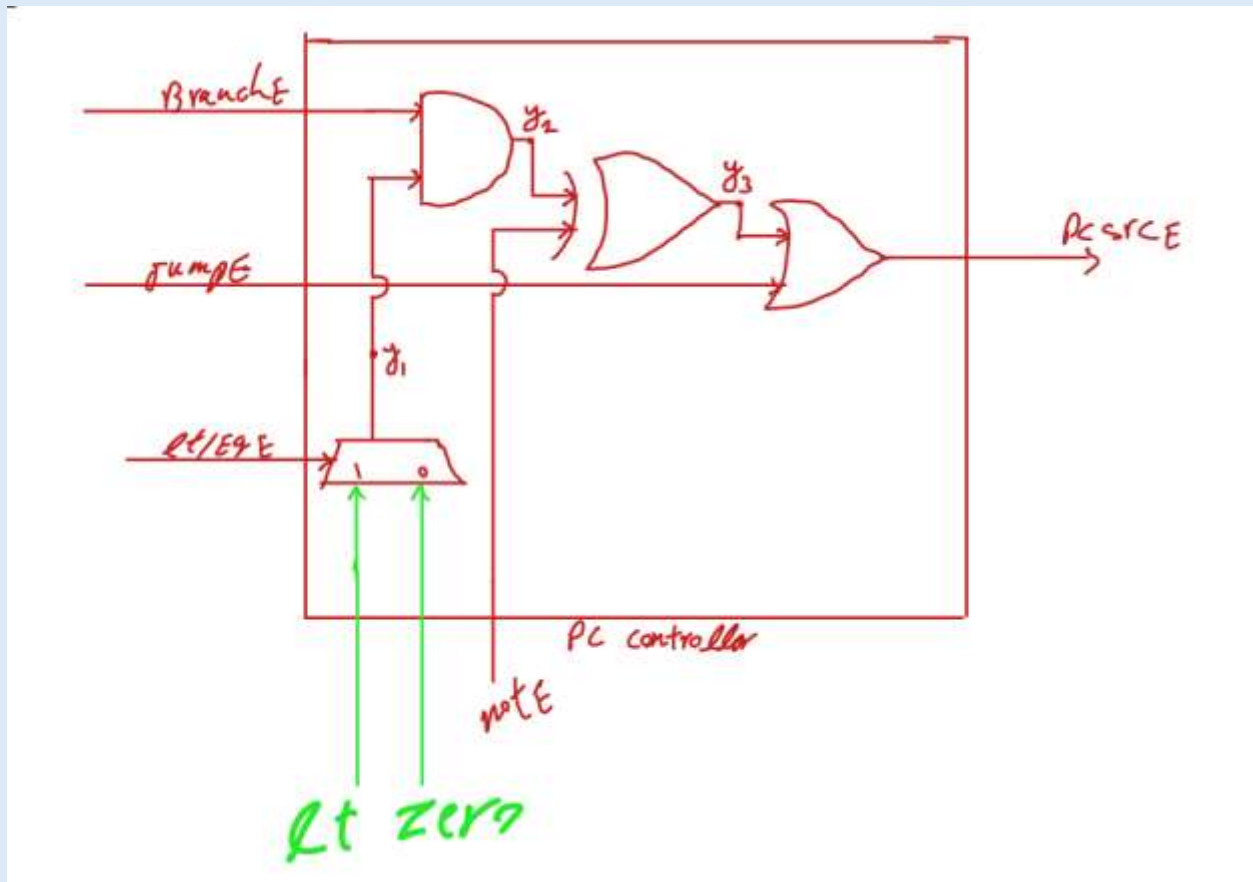


Figure 5: "\$PATH\TO\PROJECT\design\papers\CU_paper

ALU Op	Func3	Func7	Inst	ALU func	lt/eq	not	
0	_	_	lw/sw	0	_	_	
1	0	_	beq	1	0	0	
1	1	_	bnq	1	0	1	
1	4	_	blt	101	1	0	
1	5	_	bge	101	1	1	
10	0	0	add	0	_	_	
10	0	32	sub	1	_	_	
10	7	0	and	10	_	_	
10	6	0	or	11	_	_	
10	2	0	slt	101	_	_	
11	0	_	addi	0	_	_	
11	4	_	xori	100	_	_	
11	6	_	ori	11	_	_	
11	2	_	slti	101	_	_	

Figure 6: "\$PATH\TO\PROJECT\design\tables\Func_Controller_table

A	B	C	D	E	F	G	H	I	J	K	
Inst	op	ALU Op	RegWrite	ALUSrc	Branch	Jump	MemWrite	ResultSrc	ImmSrc	AddSrc	
R-T	51	10	1	0	0	0	0	0	_	_	
B-T	99	1	0	0	1	0	0	_	10	0	
I-Arithmat	19	11	1	1	0	0	0	0	0	_	
lw	3	0	1	1	0	0	0	1	0	_	
sw	35	0	0	1	0	0	1	_	1	_	
jal	111	_	1	_	_	1	0	10	11	0	
jalr	103	_	1	_	_	1	0	10	11	1	

Figure 7: "\$PATH\TO\PROJECT\design\tables\Main_Controller_table

3. Top Module Architecture and Memory Decoupling

Following the design of the datapath and control unit, and the definition of their respective I/O interfaces, we proceeded to develop the Top Module. A strategic architectural decision was made to decouple the Memory unit from the CPU core. This modular approach enhances scalability and mimics real-world hardware environments. Specific Verilog modules were dedicated to memory management and interfacing. The high-level design of the Top Module is shown below:

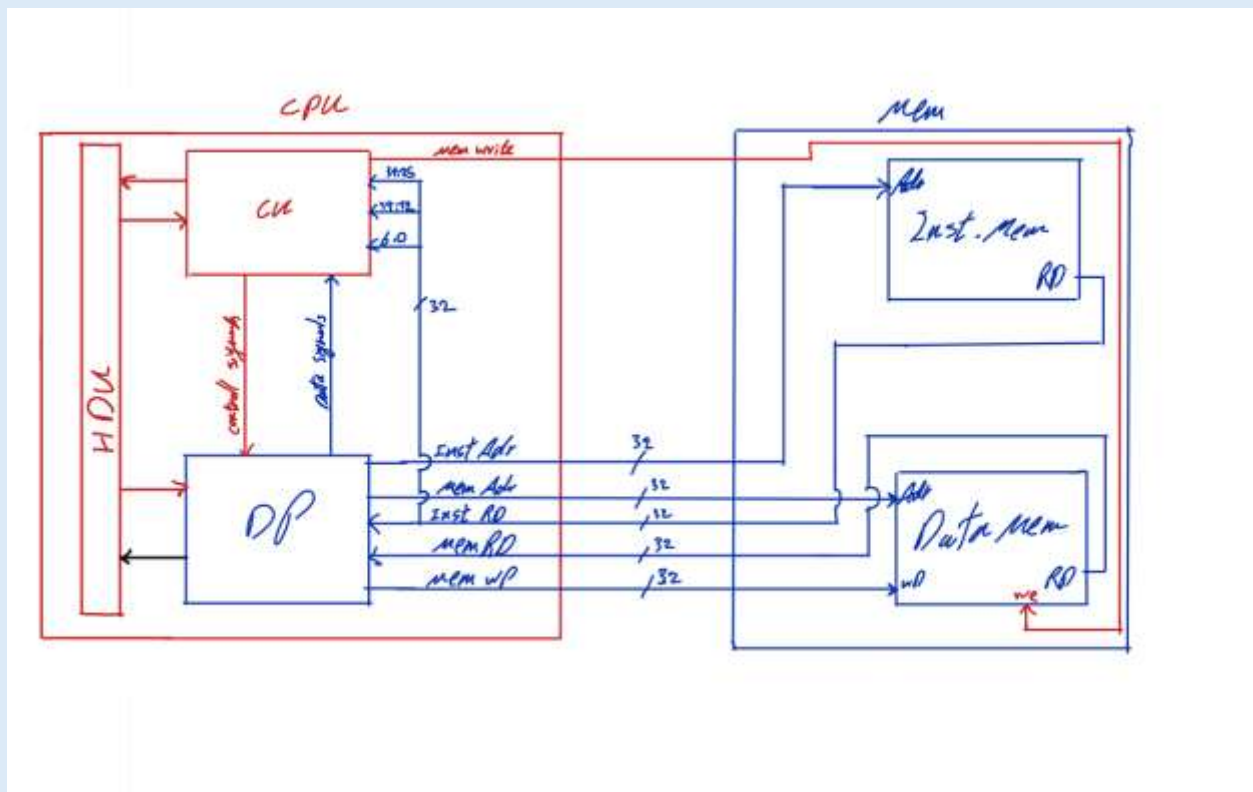


Figure 8: "\$PATH\TO\PROJECT\design\papers\Top_Module_paper

4. Implementation, Verification, and Assembly Testing

The entire system was implemented using SystemVerilog. To ensure the integrity and robustness of the design, a comprehensive Testbench was developed for the Top Module. This allowed for rigorous debugging and validation of signal timing. Finally, to verify the processor's performance against the project requirements (specifically the 10-element signed integer sorting algorithm), a series of Assembly test **cases** were executed. These test programs, used for final validation, are listed as follows:

Data Forwarding test :Program Description

This program demonstrates basic arithmetic operations using the **addi** and **add** instructions.

The registers are updated as follows:

Instruction	Result
addi x1, x0, 0	$x1 = 0$
addi x2, x1, 1	$x2 = 1$
addi x3, x2, 1	$x3 = 2$
addi x4, x3, 1	$x4 = 3$
add x4, x4, x3	$x4 = 3 + 2 = 5$
add x5, x4, x4	$x5 = 5 + 5 = 10$

Expected Result

The final register values are:

Register	Value
x1	0
x2	1
x3	2
x4	5
x5	10

This example demonstrates register initialization, data dependency between instructions, and arithmetic operations performed entirely on register values.

Simulation results:

00000000	00000000000000000000000000000000	00000000000000000000000000000000
00000002	00000000000000000000000000000001	00000000000000000000000000000010
00000004	000000000000000000000000000000101	0000000000000000000000000000001010
00000006	000000000000000000000000000000000	0000000000000000000000000000000000
00000008	000000000000000000000000000000000	0000000000000000000000000000000000
0000000a	000000000000000000000000000000000	0000000000000000000000000000000000
0000000c	000000000000000000000000000000000	0000000000000000000000000000000000
0000000e	000000000000000000000000000000000	0000000000000000000000000000000000
00000010	000000000000000000000000000000000	0000000000000000000000000000000000
00000012	000000000000000000000000000000000	0000000000000000000000000000000000
00000014	000000000000000000000000000000000	0000000000000000000000000000000000
00000016	000000000000000000000000000000000	0000000000000000000000000000000000
00000018	000000000000000000000000000000000	0000000000000000000000000000000000
0000001a	000000000000000000000000000000000	0000000000000000000000000000000000
0000001c	000000000000000000000000000000000	0000000000000000000000000000000000
0000001e	000000000000000000000000000000000	0000000000000000000000000000000000

Lw stall test :Program Description

This program demonstrates pointer chasing using the lw instruction.

The memory is initialized as follows:

Address	Value
0	4
4	8
8	12
12	16

The register x1 is initialized to 0, the address of the first memory word. Each lw instruction loads the value stored at the current address into x1. Since every loaded value is itself the address of the next element, execution follows the chain:

$0 \rightarrow 4 \rightarrow 8 \rightarrow 12 \rightarrow 16$

Expected Result

The value of x1 after each instruction is:

Instruction	x1
addi	0
1st lw	4
2nd lw	8
3rd lw	12
4th lw	16

This example demonstrates indirect memory access (pointer chasing), where each loaded value is treated as the address of the next memory location.

Simulation results:

[illegible]

Control flow test:Program Description

This program demonstrates the behavior of the ``beq`` and ``jal`` instructions in RISC-V, including conditional branching, forward and backward jumps, and the use of the link register.

Initially, both x1 and x2 are loaded from the same memory address (0), so they contain the same value. As a result, the beq instruction is taken,

skipping the next two instructions and jumping directly to the code that initializes $x1$ and $x2$.

The registers are then updated so that $x1 = 0$ and $x2 = 1$. A backward jump (`jal zero, -24`) returns execution to the `beq` instruction. This time the branch condition is false, allowing execution to continue sequentially.

The program computes $x3 = x2 + x2$, then executes a forward jump using `jal x4, 24`. This instruction stores the return address in $x4$ before jumping to another location. The destination contains `jal zero, 0`, which creates an infinite loop and terminates further program execution.

Expected Behavior

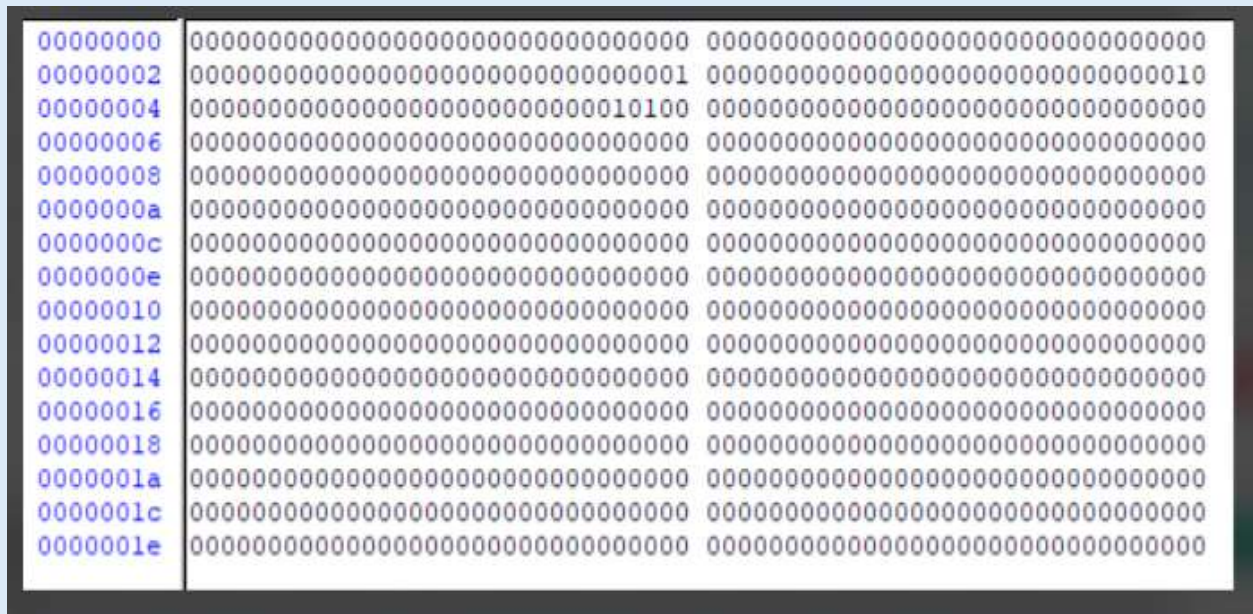
1. Load the same memory value into $x1$ and $x2$.
2. The first `beq` is taken.
3. Initialize $x1 = 0$ and $x2 = 1$.
4. Jump back to the `beq` instruction.
5. The second `beq` is not taken.
6. Compute $x3 = x2 + x2 = 2$.
7. Execute `jal x4, 24`, storing the return address in $x4$.
8. Enter an infinite loop with `jal zero, 0`.

Expected Final Register Values

Register	Value
x1	0
x2	1
x3	2
x4	20 (return address after jal)
x5	Unchanged (instruction skipped)
x6	Unchanged (never reached)
x7	Unchanged (never reached)

This program serves as a simple test for conditional branches, forward and backward jumps, and the link functionality of the jal instruction.

Simulation results:



00000000	00000000000000000000000000000000	00000000000000000000000000000000
00000002	00000000000000000000000000000001	00000000000000000000000000000010
00000004	0000000000000000000000000000010100	00000000000000000000000000000000
00000006	00000000000000000000000000000000	00000000000000000000000000000000
00000008	00000000000000000000000000000000	00000000000000000000000000000000
0000000a	00000000000000000000000000000000	00000000000000000000000000000000
0000000c	00000000000000000000000000000000	00000000000000000000000000000000
0000000e	00000000000000000000000000000000	00000000000000000000000000000000
00000010	00000000000000000000000000000000	00000000000000000000000000000000
00000012	00000000000000000000000000000000	00000000000000000000000000000000
00000014	00000000000000000000000000000000	00000000000000000000000000000000
00000016	00000000000000000000000000000000	00000000000000000000000000000000
00000018	00000000000000000000000000000000	00000000000000000000000000000000
0000001a	00000000000000000000000000000000	00000000000000000000000000000000
0000001c	00000000000000000000000000000000	00000000000000000000000000000000
0000001e	00000000000000000000000000000000	00000000000000000000000000000000

Sorting test :Description

This program implements Bubble Sort (Descending Order).

The implementation process was:

1. Writing the algorithm in C.
2. Converting it into a pseudo assembly for easier translation.
3. Converting the pseudo assembly into RISC-V assembly, where labels are replaced by instruction offsets and `j` is translated to `jal zero, offset`.

Note: Only the final RISC-V assembly is syntactically valid. The pseudo assembly is only used to illustrate the program logic.

Register Usage

Register	Purpose
s1	Base address of array A (0)
s2	i (outer loop offset)
s3	j (inner loop offset)
s4	A[j]
s5	A[j+1]
t0	Temporary register for comparisons

Program Flow

- Initialize `i = 0`.
- Execute the outer loop while `i < 40` (10×4 bytes).
- Initialize `j = 0` for each iteration.
- Execute the inner loop while `j < 36` (9×4 bytes).
- Load `A[j]` and `A[j+1]`.

- If $A[j] < A[j+1]$, swap them.
- Increase j by 4 bytes.
- Repeat until the array is sorted in descending order.
- The program ends with an infinite loop (jal zero, 0).

Sort test data mam:

00000000	00011111000111100000000000000000	0000000111010011100000001011101	00000000000000101111000100000000
00000003	000000000000000000101010100000000	00000000000000000000000000001101	00000000000000000000000000000001
00000006	11111101110000000000000000001101	111010011111111111111001111101010	1010101010101001111111111111000
00000009	10011111101111111011111000100000		
0000000c			

Sort test register file:

00000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000003	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000006	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000009	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000000c	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000000f	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000012	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000015	10011111101111111011111000100000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
00000018	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000001b	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000	00000000000000000000000000000000
0000001e	00000000000000000000000000000000	00000000000000000000000000000000		
00000021				